

1. Transaction Concepts & Lifecycle

Logical Unit of Work

Logical unit of work executing database read/write ops maintaining integrity.

- Active → Partially Committed → Committed
- Active/Partially Committed → Failed → Aborted → Terminated
- **Active:** Initial state while executing instructions.
 - **Partially Committed:** Final instruction executed; writes still in memory buffer.
 - **Committed:** Log flushed to disk; success ack sent.
 - **Failed:** Hardware/logical error occurs.
 - **Aborted:** Changes undone (rollback); restart or terminate.

2. ACID Properties Summary Matrix

Core Engine

Property	Meaning	Responsible DBMS Component
Atomicity	All operations execute or none do ("All or Nothing")	Recovery Manager (Undo Logs / WAL)
Consistency	Database transforms from one valid state to another	Application Programmer + Integrity Constraints
Isolation	Concurrent transactions run as if serial (no interference)	Concurrency Control Manager (Locks/Timestamps)
Durability	Committed changes survive any subsequent crash	Recovery Manager (Redo Logs / Stable Storage)

Key Rule: Intermediate uncommitted changes must never be visible to other transactions (Preserves Isolation).

3. Concurrency Anomalies & Isolation Violations

AKTU 10M Question

Anomaly	Conflict Pattern	Description
Dirty Read (WR)	$W1(A) \rightarrow R2(A) \rightarrow A1$	T2 uncommitted data read karta hai jo T1 abort hone par discard ho jata hai.
Lost Update (WW)	$R1(A) \dots R2(A) \dots W1(A) \dots W2(A)$	T2 T1 ke update ko bina jane overwrite kar deta hai (Blind/unaware write).

Anomaly	Conflict Pattern	Description
Unrepeatable Read (RW)	$R1(A) \rightarrow W2(A) \rightarrow R1(A)$	T1 do baar read karta hai par intermediate update ke karan value badal jati hai.
Phantom Read	$R1(\text{range}) \rightarrow \text{Ins2} \rightarrow R1(\text{range})$	Predicate query me new tuples insert hone se record count change hota hai.

4. Schedules & Serializability Foundations

Equivalence Base

- **Serial Schedule:** Transactions execute strictly one after another. Concurrency = 0, Throughput = Minimum, Consistency = 100%.
- **Non-Serial Schedule:** Operations interleave. High throughput, but anomaly prone.
- **Serializable Schedule:** A non-serial schedule equivalent to some serial schedule.
- **Conflict Operations:** Belong to T_i e T_j , access same data item Q , at least one is Write. Conflicts: $(R_i, W_j), (W_i, R_j), (W_i, W_j)$. Read-Read (R_i, R_j) is NON-conflicting.

5. Conflict Serializability & Precedence Graph

Graph Algorithm

Conflict Equivalent: $SS \equiv_C S'S$ if SS transformed into $S'S$ by swapping non-conflicting adjacent operations.

Precedence Graph $G = (V, E)$: Node per transaction T_i .

Directed Edge $T_i \rightarrow T_j$ iff T_i executes conflicting op before T_j .

AKTU Theorem: Schedule S is Conflict Serializable **IF & ONLY IF** Precedence Graph $G(S)$ contains **NO CYCLES (Acyclic)**.

Equivalent Serial Order: Derived directly using **Topological Sort** on the acyclic DAG.

6. View Serializability (VSR) & Blind Writes

NP-Complete Testing

Schedule $S \equiv_v S'$ (View Equivalent) iff 3 conditions hold:

- Initial Read:** Agar T_i $\$S\$$ me initial XX read karta hai, to S' me bhi wahi kare.
- Updated Read:** Agar T_j $\$S\$$ me T_i ka likha XX read karta hai, to S' me bhi kare.
- Final Write:** Jo transaction $\$S\$$ me final write karta hai, wahi S' me kare.

Blind Write Shortcut Rule:

- Blind Write: Transaction writes XX without reading XX first ($SW(X)$ without $RR(X)$).
- Agar schedule me **NO blind write** hai aur schedule **CSR nahi hai**, then schedule can **NEVER** be View Serializable!

Conflict Serializable (CSR) $\subset$ View Serializable (VSR)

7. Recoverability Spectrum & Cascading Aborts

Critical for Durability

Schedule Class	Formal Condition	Cascading Rollback?	AKTU Notes
Irrecoverable	T_j reads uncommitted from T_i , and $Commit(T_j) < Commit(T_i)$	Catastrophic!	Strictly disallowed in DBMS (Violates Durability).
Recoverable	$Commit(T_i) < Commit(T_j)$ whenever T_j reads uncommitted from T_i	Possible	Agar T_i abort hua to T_j bhi rollback hoga (Cascading Abort).
Cascadeless (ACA)	$Commit(T_i) < Read_j(X)$ (No uncommitted read allowed)	Zero (None)	Reads only committed data. High concurrency.
Strict	$Commit/Abort(T_i) < Read_j(X) \text{ ext}\{ \text{ and } \} Write_j(X)$	Zero (None)	Undo easily by restoring Before-Image (Old Value).

Hierarchy: Strict Schedules $\subset$ Cascadeless Schedules $\subset$ Recoverable Schedules $\subset$ All Schedules

8. Two-Phase Locking (2PL) Protocols

Serializability Guard

Locks: Shared $S(Q)$ (read-only, shared), Exclusive $X(Q)$ (read/write, exclusive).

- Growing Phase:** Transaction acquires locks, cannot release any.
- Shrinking Phase:** Transaction releases locks, cannot acquire any.
- Lock Point:** Point where the transaction acquires its final lock.

Variant	Lock Holding Rule	Guarantees
Basic 2PL	Normal 2 phases	CSR, Deadlock & Cascading possible
Strict 2PL	X-locks held till Commit	CSR + Cascadeless + Strict
Rigorous 2PL	S & X locks held till Commit	CSR + Commit Order Serializability
Conservative	Acquire all locks at start	Deadlock-Free (Low concurrency)

9. Deadlock Handling & Timestamp Schemes

Wait-Die vs Wound-Wait

Timestamp Scheme: $TS(T_i)$ timestamp at start. Smaller TS = Older.

Protocol	Rule when T_i requests lock held by T_j	Type
Wait-Die	Older waits for Younger ($TS_i < TS_j \implies \text{ext}\{\text{Wait}\}$). Younger dies ($TS_i > TS_j \implies \text{ext}\{\text{Die/Rollback}\}$).	Non-preemptive
Wound-Wait	Older wounds/preempts Younger ($TS_i < TS_j \implies \text{ext}\{\text{Rollback } T_j\}$). Younger waits ($TS_i > TS_j \implies \text{ext}\{\text{Wait}\}$).	Preemptive (Starvation Free)

Deadlock Detection: Wait-For Graph (WFG) directed edges $T_i \rightarrow T_j$. Cycle $\iff$ Deadlock. Victim selection based on minimum rollback cost.

10. Database Failures & WAL Protocol

Recovery Foundation

- Storage Hierarchy:** Volatile (RAM), Non-Volatile (Disk/SSD), Stable (Mirrored disks, survives crashes).
- Failure Types:** Transaction Failure (logical/system), System Crash (RAM loss), Disk Failure (head crash).
- Write-Ahead Logging (WAL):** Log record $\langle T_i, X, V_{old}, V_{new} \rangle$ must be written to stable storage BEFORE database item X is updated on disk.

Technique	Disk Write Timing	Recovery Action
Deferred Mod	Only after Commit	REDO only (No UNDO required)
Immediate Mod	Anytime (Uncommitted)	Both UNDO & REDO required

11. Checkpoints & Shadow Paging

Performance Boost

Checkpoints: Limits log scan overhead on system crash.

- Log Record: $\langle \text{checkpoint } L \rangle$ where L = active transactions list.
- Scan backward to last checkpoint.
 - T_i with Commit/Abort $\rightarrow$ REDO list (Forward scan).
 - T_i active without Commit $\rightarrow$ UNDO list (Backward scan).

Shadow Paging: Maintains Current Page Table in RAM, Shadow Page Table on disk. Commit = atomically pointing to current page table. No logs needed, but page fragmentation occurs.

12. Distributed Databases (DDBMS) & Directory Systems

Architecture & Protocols

Data Fragmentation:

- Horizontal:** Rows split by predicate $\sigma_p(R)$. Reconstruct: $R_1 \cup R_2$.
- Vertical:** Columns split by projection $\pi(R)$ keeping PK. Reconstruct: $R_1 \bowtie R_2$.
- Hybrid:** Nested horizontal & vertical.

Correctness Rules:

- Completeness:** Har tuple/attribute fragment me hona chahiye.
- Reconstruction:** Lossless join/union se original relation restore hona chahiye.
- Disjointness:** Non-key attributes overlap nahi hone chahiye.

Directory & 2PC Protocol:

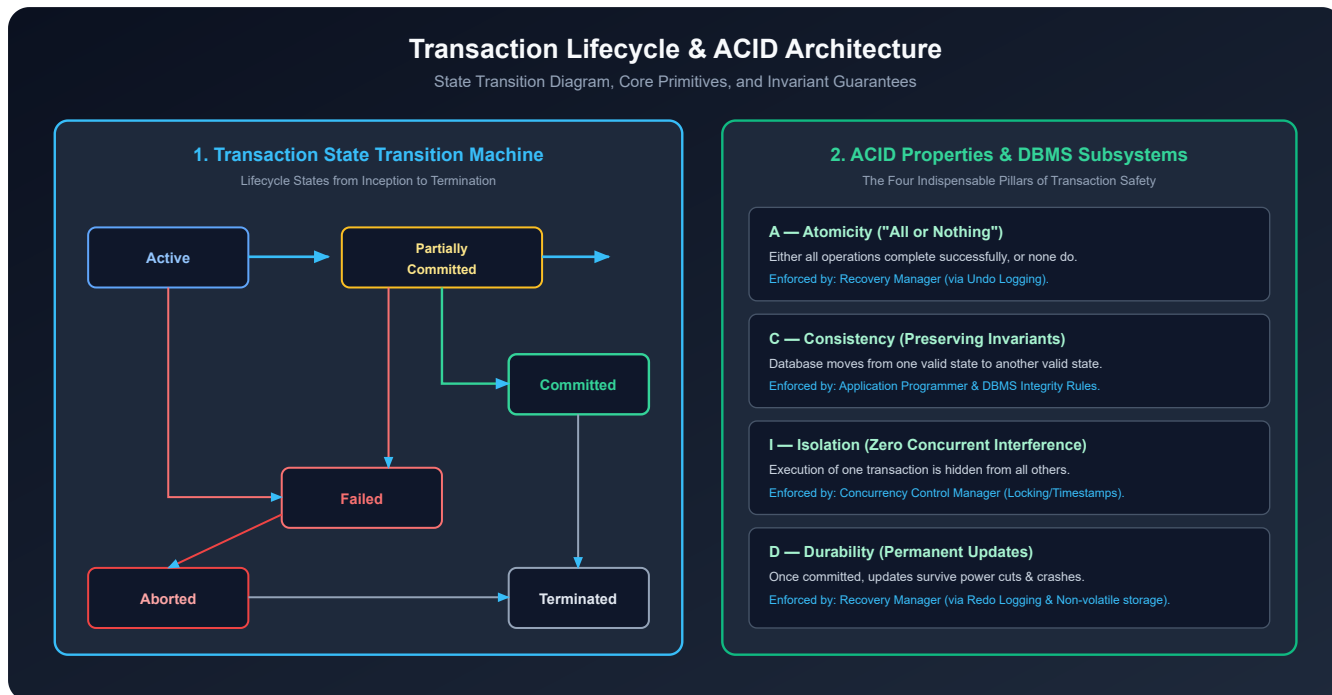
- LDAP:** Lightweight Directory Access Protocol (X.500 based, DIT tree, read-optimized).
- Two-Phase Commit (2PC):**
Phase 1: Prepare & Vote (Commit/Abort).
Phase 2: Global Decision (Commit/Abort).

13. Master AKTU Exam Problem Solving Flowchart & Checklist

100% Exam Formulae

Step	Question Type	Actionable Solution Algorithm	Outcome / Formula
1	Conflict Serializability (CSR)	Identify conflicting pairs $(R_i, W_j), (W_i, R_j), (W_i, W_j)$ on same variable. Draw edge $T_i \rightarrow T_j$.	Acyclic $\implies$ CSR. Cyclic $\implies$ Not CSR. Topological sort = Serial order.
2	View Serializability (VSR)	Agar CSR hai $\implies$ Directly VSR! Agar Not CSR hai, check for Blind Writes.	Agar koi blind write nahi hai $\implies$ Directly NOT VSR! If blind write present, test equivalence.
3	Recoverability Check	Locate every Read-From pair: $W_i(X) \dots R_j(X)$ where T_j reads uncommitted X from T_i .	If $C_i < C_j \implies$ Recoverable. If $C_j < C_i \implies$ Irrecoverable. If $C_i < R_j(X) \implies$ Cascadeless.
4	2PL Schedule Verification	Trace every lock acquisition and release. Check if any lock acquired AFTER a release.	If lock acquired after unlock $\implies$ Violates 2PL. If all X-locks held till commit $\implies$ Strict 2PL.
5	Log Recovery (Undo/Redo)	Find checkpoint. Transactions with 'commit' record $\rightarrow$ 'REDO'. Transactions without 'commit' $\rightarrow$ 'UNDO'.	Execute backward for 'UNDO' (restore V_{old}) and forward for 'REDO' (apply V_{new}).

Module 01: Transaction Concepts & ACID Properties



> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Transaction Definition, Atomic Operations ('Read'/'Write'), Transaction State Transition Machine, ACID Properties & Their DBMS Enforcement Subsystems, Failure Types.

1. What is a Transaction?

Database Management Systems me ek **Transaction** logically related operations ka ek single collection (unit of work) hota hai jo ya toh poora execute hota hai ya bilkul bhi nahi.

Formal Definition: A transaction is a single logical unit of database processing that accesses and possibly updates various data items.

Real-world example: **Bank Fund Transfer**

- Account \$A\$ se Account \$B\$ me \\$500 transfer karna:

1. Read(A)
2. $A = A - 500$
3. Write(A)
4. Read(B)
5. $B = B + 500$
6. Write(B)
7. Commit

- Agar step 3 ke baad power chali jaye ya server crash ho jaye, toh system ko aisi state me nahi chhod sakte jahan \$A\$ se paise cut gaye par \$B\$ ko nahi mile!

2. Fundamental Operations in a Transaction

Har high-level database query physically do primitive operations me translate hoti hai:

1. Read(X):

- Disk/Database se data item \$X\$ ko fetch karke transaction ke local memory buffer me copy karta hai.

2. Write(X):

- Transaction ke local memory buffer se updated value ko database buffer/disk par write-back karta hai.

3. Transaction State Transition Machine

Ek transaction apne execution lifecycle ke dauran specified formal states se guzarta hai:

,



Terminated

State Explanations:

- 1. Active:** Initial state jahan transaction start hota hai aur operations execute kar raha hota hai.
- 2. Partially Committed:** Transaction ka aakhiri instruction execute ho chuka hai, par updates abhi bhi main memory buffers me hain (disk par permanently flush nahi hue).
- 3. Committed:** Transaction ke sabhi updates permanently non-volatile storage (disk) par save ho chuke hain. Ab transaction ko rollback nahi kiya ja sakta.
- 4. Failed:** Normal execution ke waqt agar hardware crash, logical bug, ya system error aaye toh transaction Failed state me chala jaata hai.
- 5. Aborted:** Failed transaction ke saare changes undo (rollback) karke database ko original initial consistent state me restore kiya jaata hai.
- 6. Terminated:** Transaction system chhod kar exit kar chuka hai (either successfully committed or cleanly aborted).

4. ACID Properties & DBMS Enforcement Architecture

Database consistency maintain karne ke liye Jim Gray dwara define kiye gaye **ACID Properties** mandatory hain:

Property	Core Guarantee	Real-World Scenario	Responsible DBMS Subsystem
Atomicity	***"All or Nothing"*** — Ya toh saare steps execute honge ya ek bhi nahi.	Agar paisa debit hua toh credit zaroor hoga; crash hone par pura debit undo ho jayega.	**Recovery Manager** (Undo logging)
Consistency	**Preserving Invariants** — Database transaction se pehle aur baad me legal valid rules satisfy karega.	Total sum of money $$(A + B)$$ transfer ke pehle aur baad exactly same rahegi.	**Application Developer** + **Integrity Subsystem**
Isolation	**Concurrent Independence** — Do parallel transactions ek doosre ke intermediate states ko dekh nahi sakte.	User $$(T_1)$$ ke debit hone ke turant baad par credit se pehle User $$(T_2)$$ balance check kare toh use intermediate galat data na dikhe.	**Concurrency Control Manager** (Locks & Timestamps)
Durability	**Permanence** — Commit message aane ke baad system crash ho jaye toh bhi data kabhi kho nahi sakta.	Flight ticket book ho gayi toh server restart hone par bhi seat booked hi rahegi.	**Recovery Manager** (Redo logging & WAL)

5. Architectural Diagram

6. Solved AKTU Examination Question (10 Marks)

Question: Explain the ACID properties of a transaction. What are the various states through which a transaction passes during its lifecycle? Explain with a neat state transition diagram. *(AKTU 2021-22, 2023-24)*

Model Answer Summary:

1. State definition of Transaction as single logical unit of work with Bank transfer example.
2. Draw the 6-state transition diagram (Active, Partially Committed, Committed, Failed, Aborted, Terminated).
3. Explain conditions for transitioning between states:

- Partial Commit \$

ightarrow\$ Commit: When log records and dirty pages are safely written to non-volatile disk.

- Active \$

ightarrow\$ Failed: System error, divide-by-zero, power loss.

- Failed \$

ightarrow\$ Aborted: Rollback execution restoring before-images from undo log.

4. Tabulate ACID properties with their respective DBMS manager components.

Module 02: Concurrency Problems & Anomalies

Concurrency Anomalies & Multi-User Hazards

The Four Classic Anomalies Occurring in Uncontrolled Interleaved Executions

1. Dirty Read Problem (Reading Uncommitted Data)

Write-Read (W-R) Conflict Hazard

T1 modifies data item A (W1(A)). T2 reads updated A before T1 commits.
T1 then FAILS and ABORTS (Rollback to original value)!

Consequence: T2 has executed based on a "dirty" value that never existed!

Timeline: W1(A) → R2(A) → T1 Aborts [Dirty Read by T2!]

Prevention: Enforce READ COMMITTED or REPEATABLE READ isolation.

2. Lost Update Problem (Blind Overwrites)

Write-Write (W-W) Conflict Hazard

Both T1 and T2 read same data item A. T1 updates A and writes it back.
T2 overwrites A with its own calculation without incorporating T1's change!

Consequence: T1's modification is completely erased and LOST!

Timeline: R1(A) → R2(A) → W1(A) → W2(A) [T1 update lost!]

Prevention: Strict Two-Phase Locking (Exclusive Locks on Write).

3. Unrepeatable Read (Fuzzy Read)

Read-Write (R-W) Conflict Hazard

T1 reads data item A. Meanwhile, T2 updates A and commits.

T1 reads A again within same transaction and observes a DIFFERENT value!

Consequence: Transaction observes contradictory data during its run.

Timeline: R1(A) → W2(A) → Commit2 → R1(A) [Differs from 1st read!]

Prevention: REPEATABLE READ isolation level or shared read-locks.

4. Phantom Read Problem (Tuple Insertion Hazard)

Range Query Predicate Interference

T1 queries a set of rows matching predicate (eg: salary > 50k; gets 5 rows).

T2 inserts a new row matching predicate and commits.

T1 repeats same range query and finds 6 rows! A "phantom" row appeared!

Timeline: Query1(range) → Insert2 → Commit2 → Query1(range)

Prevention: SERIALIZABLE isolation level (Index range / Predicate locking).

> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Why Concurrency is Needed, Concurrency Hazards, Dirty Read Problem (WR), Lost Update Problem (WW), Unrepeatable Read Problem (RW), Phantom Read Problem, SQL Isolation Levels.

1. Why Do We Need Concurrent Transaction Execution?

Single-user database me transactions ek ke baad ek (serially) run hoti hain, jisse koi anomaly nahi hoti. Lekin enterprise DBMS (e.g., IRCTC, Banking, Amazon) me hazaron users ek saath access karte hain.

Benefits of Concurrency:

1. **Higher Throughput:** Unit time me zyada transactions complete hoti hain.
2. **Reduced Waiting / Response Time:** Chhote transactions lambe transactions ke peeche block nahi hote.
3. **High Resource Utilization:** CPU computation aur Disk I/O parallel me overlap ho sakte hain.

Lekin agar concurrency ko bina kisi protocol ke allow kiya jaye, toh database me **4 classical anomalies** paida hoti hain!

2. The Four Classical Concurrency Anomalies

2.1 Dirty Read Problem (Write-Read Conflict)

Scenario: Transaction \$T_1\$ data item \$A\$ ko update karta hai (\$W_1(A)\$). Abhi \$T_1\$ ne commit nahi kiya hai.

Us uncommitted data ko transaction \$T_2\$ read kar leta hai (\$R_2(A)\$).

Uske baad kisi error ki wajah se \$T_1\$ **Abort** ho jaata hai aur rollback kar diya jaata hai!

Hazard: T_2 ne ek aisi value par calculation kar li jo physically database me kabhi exist hi nahi karti thi!

T1 T2

| |

Read(A) |

$A = A + 100$ |

Write(A) |

| Read(A) <-- DIRTY READ! (Uncommitted data)

| Write(A)

[ABORT / ROLLBACK] |

,

2.2 Lost Update Problem (Write-Write Conflict)

Scenario: Do transactions T_1 aur T_2 simultaneously data item A ki initial value ko read karti hain ($A = 1000$).

T_1 ne $A = A - 100$ kiya aur write kar diya ($A = 900$).

Par turant T_2 ne $A = A + 200$ calculate kiya aur bina T_1 ke update ko consider kiye write kar diya ($A = 1200$).

Hazard: T_1 ka subtraction update permanently **Lost** ho gaya! Final balance 1100 hona chahiye tha, par database me 1200 reh gaya!

,

T1 T2

| |

Read(A) [1000] |

| Read(A) [1000]

$A = A - 100$ |

Write(A) [900] |

| $A = A + 200$

| Write(A) [1200] <-- T_1 's update is LOST!

,

2.3 Unrepeatable Read Problem (Read-Write Conflict)

Scenario: Transaction \$T_1\$ data item \$A\$ ko read karta hai (\$A = 500\$).

Tabhi \$T_2\$ aata hai, \$A\$ ko modify karta hai (\$A = 700\$) aur **Commit** kar deta hai.

Transaction \$T_1\$ apne execution ke dauran dobara \$A\$ ko read karta hai.

Hazard: Ek hi transaction ke andar user ko do alag-alag values milti hain! (\$500\$ and \$700\$). Is anomaly ko **Fuzzy Read** bhi kehte hain.

2.4 Phantom Read Problem (Dynamic Range Hazard)

Scenario: \$T_1\$ ek range query run karta hai: `SELECT * FROM Emp WHERE Salary > 50000` aur use 10 records milte hain.

Is dauran \$T_2\$ ek naya employee insert karta hai jiska salary \$60,000\$ hai aur commit kar deta hai.

\$T_1\$ jab dobara wahi query run karta hai, toh use 11 records milte hain!

Hazard: Naya tuple bina kisi update ke "bhoot" (phantom) ki tarah appear ho gaya!

3. SQL-92 Standard Isolation Levels

ANSI/SQL standard ne concurrency anomalies ko balance karne ke liye 4 isolation levels define kiye hain:

Isolation Level	Dirty Read Allowed?	Unrepeatable Read Allowed?	Phantom Read Allowed?	Performance Overhead
Read Uncommitted	Yes ()	Yes	Yes	Lowest overhead
Read Committed	**No (\$\checkmark\$)**	Yes	Yes	Default in Oracle / PostgreSQL
Repeatable Read	**No (\$\checkmark\$)**	**No (\$\checkmark\$)**	Yes	Default in MySQL InnoDB
Serializable	**No (\$\checkmark\$)**	**No (\$\checkmark\$)**	**No (\$\checkmark\$)**	Highest isolation / Slowest

4. Architectural Diagram

5. AKTU Semester Numerical & Concept Question

Question: What are the various problems that arise due to concurrent execution of transactions? Explain Dirty Read and Lost Update with suitable transaction schedules. *(AKTU 2020-21, 2022-23)*

Solution Points:

1. Draw the comparative schedule table showing interleaving of \$T_1\$ and \$T_2\$.
2. Write initial values (e.g., \$A = 1000\$).
3. Trace steps showing how \$W_1(A)\$ followed by \$R_2(A)\$ creates dirty read when \$T_1\$ issues ABORT.
4. Trace steps showing how concurrent reads followed by overwriting writes result in lost update.

5. Provide solution: Use Strict 2PL and write-locks.

Module 03: Schedules & Serializability Foundations

Schedules & Serializability Foundations

Serial Schedules, Non-Serial Interleaving, and the Definition of Equivalence

1. Schedule Taxonomy & Classification

Execution Sequences of Interleaved Operations

Serial Schedule (Zero Interleaving)

Transactions execute strictly one after another.
Next transaction starts ONLY after current commits.
✓ 100% Consistent by definition! ✗ Poor throughput.

Non-Serial Schedule (Concurrent Interleaving)

Operations of multiple transactions are interleaved.
Maximizes CPU/Disk overlap and system responsiveness.
⚠ May lead to severe inconsistency if not serializable!

Permutations & Combinatorics Formula:

For n transactions: Total Serial Schedules = $n!$
For transactions with $n_1, n_2, \dots, n_k$ operations:
Total Non-Serial Schedules = $(n_1 + n_2 + \dots + n_k)! / (n_1! \cdot n_2! \cdot \dots \cdot n_k!)$

2. Concept of Serializability

The Benchmark for Concurrent Schedule Correctness

Formal Serializability Definition:

A non-serial schedule S is defined as Serializable if its final database state is EQUIVALENT to the outcome of some serial schedule S_{serial} of the same transactions.
Offers speed of interleaving with correctness of serial order!

Two Formal Types of Serializability:

- Conflict Serializability: Swapping non-conflicting ops.
Fast polynomial test $O(V + E)$ via Precedence Graph!
- View Serializability: Matching initial, updated & final reads.
NP-Complete problem! Handled via blind writes.

Mathematical Hierarchy:

Conflict Serializable $\subset$ View Serializable $\subset$ All Schedules
Every Conflict Serializable schedule is ALWAYS View Serializable.
View Serializable schedule without Blind Writes = Conflict Serializable!

> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Definition of Schedule, Serial vs Non-Serial Schedules, Combinatorics of Schedules, Serializability as the Benchmark of Correctness, Conflict vs View Serializability Hierarchy.

1. What is a Schedule?

Jab multiple transactions system me ek saath run hoti hain, toh unke operations (read, write, commit, abort) jis chronological sequence (time order) me CPU par execute hote hain, use **Schedule** (ya History) kehte hain.

Formal Definition: A schedule \$\$\$ of n transactions $\{T_1, T_2, \dots, T_n\}$ is an ordering of the operations of the transactions such that for each transaction T_i , the operations of T_i in \$\$\$ appear in the same order as in T_i .

2. Classification of Schedules

2.1 Serial Schedule

Ek transaction ke saare operations ek saath execute hote hain. Doosra transaction tabhi shuru hota hai jab pehla **Commit** ho chuka ho.

Example: T_1 executes completely \$

$\rightarrow$ then T_2 executes completely (T_1

$\rightarrow T_2$).

Properties:

- Database hamesha **100% consistent** rehta hai (no concurrency hazards).
- CPU utilization bohot bekar hoti hai; response time bohot high hota hai (Zero concurrency).

2.2 Non-Serial (Interleaved) Schedule

Transactions ke operations ek doosre ke beech me mix (interleave) hokar run hote hain.

Properties:

- High system throughput aur fast response time.
- Lekin agar control na kiya jaye, toh inconsistent database ban sakta hai!

2.3 Combinatorics Formula (AKTU Objective & Gate Questions)

Agar relation me n transactions hain:

1. Total Number of Serial Schedules:

$$\text{Total Serial} = n!$$

2. Total Possible Concurrent Schedules:

Agar transaction T_1 me n_1 operations hain, T_2 me n_2 operations hain, ..., T_k me n_k operations hain:

$$\text{Total Schedules} =$$

$$\frac{(n_1 + n_2 + \dots + n_k)!}{n_1! \times n_2! \times \dots \times n_k!}$$

3. The Concept of Serializability

Golden Rule of Serializability:

Ek concurrent (non-serial) schedule SS tabhi **Correct** maana jaata hai jab uska final effect kisi ek **Serial Schedule** ke barabar ho!

Is property ko **Serializability** kehte hain.

Serial schedule benchmark hota hai.

Agar non-serial schedule serial schedule ke barabar outcome de raha hai, toh hume concurrency ki **high speed** bhi mil gayi aur serial execution ki **100% consistency** bhi mil gayi!

4. Types of Serializability

Serializability ko verify karne ke do theoretical approaches hain:

1. Conflict Serializability:

- Non-conflicting adjacent operations ko swap karke serial schedule me convert karna.
- Tested using **Precedence Graph (Cycle Detection)** in polynomial time $O(V + E)$.

2. View Serializability:

- Har transaction ke initial read, intermediate updated read, aur final write ko serial schedule se match karna.
- Testing view serializability is **NP-Complete!**

4.1 Master Inclusion Hierarchy

$$\mathbf{\text{Conflict Serializable} \subseteq \text{View Serializable} \subseteq \text{All Valid Schedules}}$$

Har Conflict Serializable schedule **by default View Serializable hota hai.**

Ek View Serializable schedule jo conflict serializable nahi hai, usme compulsory roop se **Blind Writes** ($W(A)$ without prior $R(A)$) present honge!

5. Architectural Diagram

Module 04: Conflict Serializability & Precedence Graph

1. The 3 Conditions for Conflicting Operations

Two operations l_i and l_j in schedule S conflict iff:

1. They belong to **DIFFERENT** transactions ($T_i \neq T_j$).
2. They access the **EXACT SAME** data item (eg: both on A).
3. **At least ONE** of the two operations is a **WRITE** operation!

Conflicting Pairs Matrix:

- Read-Read ($R_i(A), R_j(A)$): **NON-CONFLICTING** (Can swap safely!)
- Read-Write ($R_i(A), W_j(A)$): **CONFLICTING** (Cannot swap!)
- Write-Read ($W_i(A), R_j(A)$): **CONFLICTING** (Cannot swap!)
- Write-Write ($W_i(A), W_j(A)$): **CONFLICTING** (Cannot swap!)

Swapping non-conflicting adjacent operations preserves final state.

Conflict Equivalence & Serializability:

S is conflict serializable iff it can be transformed into a serial schedule by repeatedly swapping non-conflicting operations.

2. Precedence Graph (Serialization Graph)

The Definite Theorem: Acyclic Graph $\Leftrightarrow$ Conflict Serializable

Graph Construction Rules:

- Vertices (V): All participating transactions $\{T_1, T_2, \dots, T_n\}$.
- Directed Edge ($T_i \rightarrow T_j$): Drawn if T_i executes a conflicting operation **BEFORE** T_j executes its conflicting operation on same item.
(Look for R_i-W_j , W_i-R_j , or W_i-W_j where T_i happens before T_j).

The Core Mathematical Theorem:

1. If Graph is **ACYCLIC** (No cycles) $\Rightarrow$ **CONFLICT SERIALIZABLE!**
2. If Graph has a **CYCLE** $\Rightarrow$ **NOT Conflict Serializable!**

Complexity: $O(V + E)$ using Depth-First Search (DFS).

Finding Equivalent Serial Order:

Apply **TOPOLOGICAL SORTING** on the DAG (Directed Acyclic Graph):

- Find node with In-Degree = 0 (Start transaction).
- Remove node, decrement neighbor in-degrees, repeat!

Yields the exact serial order. Eg: $T_1 \rightarrow T_3 \rightarrow T_2$.

> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Conflicting Operations Criteria, Conflict Equivalence, Precedence Graph (Serialization Graph) Algorithm, Cycle Detection Theorem, Topological Sorting for Serial Order, Solved Gateway AKTU Numericals.

1. What are Conflicting Operations?

Schedule ke do operations S_{l_i} aur S_{l_j} tabhi **Conflicting Operations** kehlate hain jab wo neeche di gayi teeno conditions satisfy karein:

1. Dono operations **alag-alag transactions** ke hon (T_i e T_j).
2. Dono operations **exact same data item** par execute ho rahe hon (e.g., both on variable A).
3. Dono me se **kam se kam ek operation WRITE** ho!

1.1 Conflicting Operations Matrix

Operation Pair	Conflict?	Reasoning	Can Swap?
$R_i(A)$ and $R_j(A)$	**NO**	Read operations do not modify data state.	**YES**
$R_i(A)$ and $W_j(A)$	**YES**	Read-Write (RAW) conflict. Changing order changes read value.	**NO**
$W_i(A)$ and $R_j(A)$	**YES**	Write-Read (WAR) conflict.	**NO**
$W_i(A)$ and $W_j(A)$	**YES**	Write-Write (WAW) conflict. Changing order changes final state.	**NO**
$R_i(A)$ and $W_j(B)$	**NO**		

e B\$). | **YES** |

2. Conflict Equivalence & Conflict Serializability

Conflict Equivalent: Do schedules S_1 aur S_2 conflict equivalent tab hote hain jab S_1 ko non-conflicting adjacent operations ko swap karke S_2 me transform kiya ja sake.

Conflict Serializable: Ek schedule S conflict serializable tab hota hai jab wo kisi ek Serial Schedule ke conflict-equivalent ho!

3. Precedence Graph (Serialization Graph) Algorithm

Exam me haath se swap karte-karte confuse hone ki jagah, hum **Precedence Graph (Directed Graph $G = (V, E)$)** banate hain:

,

Algorithm: Test_Conflict_Serializability(S)

1. Vertices (V): Schedule me jitni transactions hain, unke nodes banao: $\{T_1, T_2, \dots, T_n\}$.
2. Edges (E): For every conflicting pair (l_i, l_j) on data item A :

If l_i belongs to T_i , l_j belongs to T_j , and l_i occurs BEFORE l_j in schedule S :

Draw a directed edge $T_i \rightarrow T_j$.

3. Cycle Detection:

Run Cycle Detection Algorithm (DFS / In-degree check).

- If Graph contains a CYCLE:

=> Schedule is NOT Conflict Serializable!

- If Graph has NO CYCLES (Acyclic DAG):

=> Schedule IS Conflict Serializable!

,

3.1 Topological Sort for Finding Equivalent Serial Schedule

Agar graph acyclic hai, toh equivalent serial order nikalne ke liye:

1. Sabhi vertices ka **In-Degree** (incoming edges count) calculate karo.
2. Jiska In-Degree = 0 ho, use serial order me pehle likho aur graph se uske outgoing edges hata do.
3. Bachi hui nodes me step repeat karo.
4. Output order will be the **Equivalent Serial Schedule**!

4. Architectural Diagram

5. Gateway Classes Solved AKTU Examination Numerical

Question: Check whether the given schedule S is conflict serializable or not. If yes, find the equivalent serial schedule:

S : $R_1(X); \text{ \quad } R_2(Z); \text{ \quad } R_1(Z); \text{ \quad } R_3(X); \text{ \quad } R_3(Y); \text{ \quad } W_1(X); \text{ \quad } W_3(Y); \text{ \quad } R_2(Y);$
 $\text{ \quad } W_2(Z); \text{ \quad } W_2(Y)$

Step 1: Identify Participating Transactions

Transactions = $\{T_1, T_2, T_3\}$. Draw nodes T_1 , T_2 , T_3 .

Step 2: Trace Conflicting Operation Pairs in Chronological Order

1. On Item X :

- $R_3(X)$ is followed by $W_1(X) \implies \mathbf{T_3 \rightarrow T_1}$.

2. On Item Y :

- $W_3(Y)$ is followed by $R_2(Y) \implies \mathbf{T_3 \rightarrow T_2}$.

- $W_3(Y)$ is followed by $W_2(Y) \implies \mathbf{T_3 \rightarrow T_2}$ (Already exists).

- $R_2(Y)$ is followed by $W_2(Y) \implies$ Same transaction T_2 (Ignored).

3. On Item Z :

- $R_1(Z)$ is followed by $W_2(Z) \implies \mathbf{T_1 \rightarrow T_2}$.

- $R_2(Z)$ is followed by $W_2(Z) \implies$ Same transaction T_2 (Ignored).

Step 3: Draw Directed Edges

The edges in graph G are:

T_3

$\rightarrow T_1$

T_3

$\rightarrow T_2$

T_1

$\rightarrow T_2$

,

$T_3 \longrightarrow T_1$

$\mid \mid$

$\mid \mid$

$\longleftarrow T_2 \longleftarrow$

,

Step 4: Check for Cycles

Paths: T_3

$\rightarrow T_1$

$\rightarrow T_2$, and T_3

$\rightarrow T_2$.

Is there any back-edge or loop? **NO! Graph is strictly ACYCLIC.**

Step 5: Equivalent Serial Order via Topological Sort

In-degree of $T_3 = 0 \implies$ Starts with $\mathbf{T_3}$.

Remove T_3 . Remaining: In-degree of $T_1 = 0 \implies$ Next is $\mathbf{T_1}$.

Remaining: $\mathbf{T_2}$.

Equivalent Serial Schedule: $\mathbf{T_3}$

$\rightarrow T_1$

$\rightarrow T_2$

Verdict: Schedule SS is **Conflict Serializable!**

Module 05: View Serializability & Blind Writes



2. View Serializability Definition

A non-serial schedule S is **View Serializable** if it is **view equivalent** to some serial schedule of the same transactions.

3. The Crucial Role of "Blind Writes"

Blind Write: Jab koi transaction kisi variable A par write karta hai bina use pehle read kiye ($W(A)$ without preceding $R(A)$).

The Blind Write Shortcut Theorem (AKTU Exam Lifesaver):

1. Agar kisi schedule me **koi bhi Blind Write nahi hai**, aur uska Precedence Graph **cyclic** hai:

$$\text{No Blind Writes} + \text{Cycle in Graph} \implies \text{NOT View Serializable}$$

(Exam me time waste karne ki zaroorat nahi, direct "Not View Serializable" likhein!)

2. Agar kisi schedule me Precedence Graph me cycle hai, lekin **Blind Write present hai**, tab wo schedule **View Serializable ho bhi sakta hai aur nahi bhi!** (Polygraph test / manual view check required).

4. Conflict vs View Serializability Comparison

Feature	Conflict Serializability	View Serializability
Testing Complexity	Polynomial $O(V + E)$ (Precedence Graph)	NP-Complete ($O(n!)$ serial permutations)
Swapping Operations	Swaps adjacent non-conflicting ops	Matches data read/write views
Condition Strictness	More Strict	Relaxed (allows blind write overwrites)
Mathematical Relation	$\text{Conflict Serializable} \subseteq \text{View Serializable}$	Every conflict serializable is view serializable

5. Architectural Diagram

6. Gateway Classes Solved AKTU Examination Problem

Problem: Test whether the following schedule S is View Serializable:

$$S: R_1(A); \quad W_2(A); \quad W_1(A); \quad W_3(A)$$

Step 1: Precedence Graph for Conflict Serializability

$$R_1(A) \text{ followed by } W_2(A) \implies T_1$$

$$\rightarrow T_2.$$

$W_2(A)$ followed by $W_1(A)$ implies T_2

$\rightarrow T_1$.

Cycle detected: T_1

$\rightarrow T_2$!

Verdict 1: Schedule SS is **NOT Conflict Serializable**.

Step 2: Check for Blind Writes

In T_2 : It executes $W_2(A)$ without reading A implies **Blind Write present!**

In T_3 : It executes $W_3(A)$ without reading A implies **Blind Write present!**

Because blind writes exist, SS might still be View Serializable!

Step 3: Test View Equivalence with Candidate Serial Orders

Transactions are $\{T_1, T_2, T_3\}$.

Possible serial orders where T_3 is last (since T_3 is the final writer in SS):

Candidate 1: T_2

$\rightarrow T_1$

$\rightarrow T_3$

Let us test rules for Candidate 1 (T_2

$\rightarrow T_1$

$\rightarrow T_3$):

1. Initial Read:

- In SS : T_1 reads initial A .

- In T_2

$\rightarrow T_1$

$\rightarrow T_3$: T_2 only writes A , then T_1 reads A (Wait, T_1 reads T_2 's write, NOT initial!). Fails!

Candidate 2: Try T_1

$\rightarrow T_2$

$\rightarrow T_3$:

1. **Initial Read:** In SS , T_1 reads initial A . In Candidate 2, T_1 runs first, so T_1 reads initial A . **(Satisfied!)**

2. **Updated Read:** In SS , no transaction reads any updated value. In Candidate 2, no transaction reads any updated value. **(Satisfied!)**

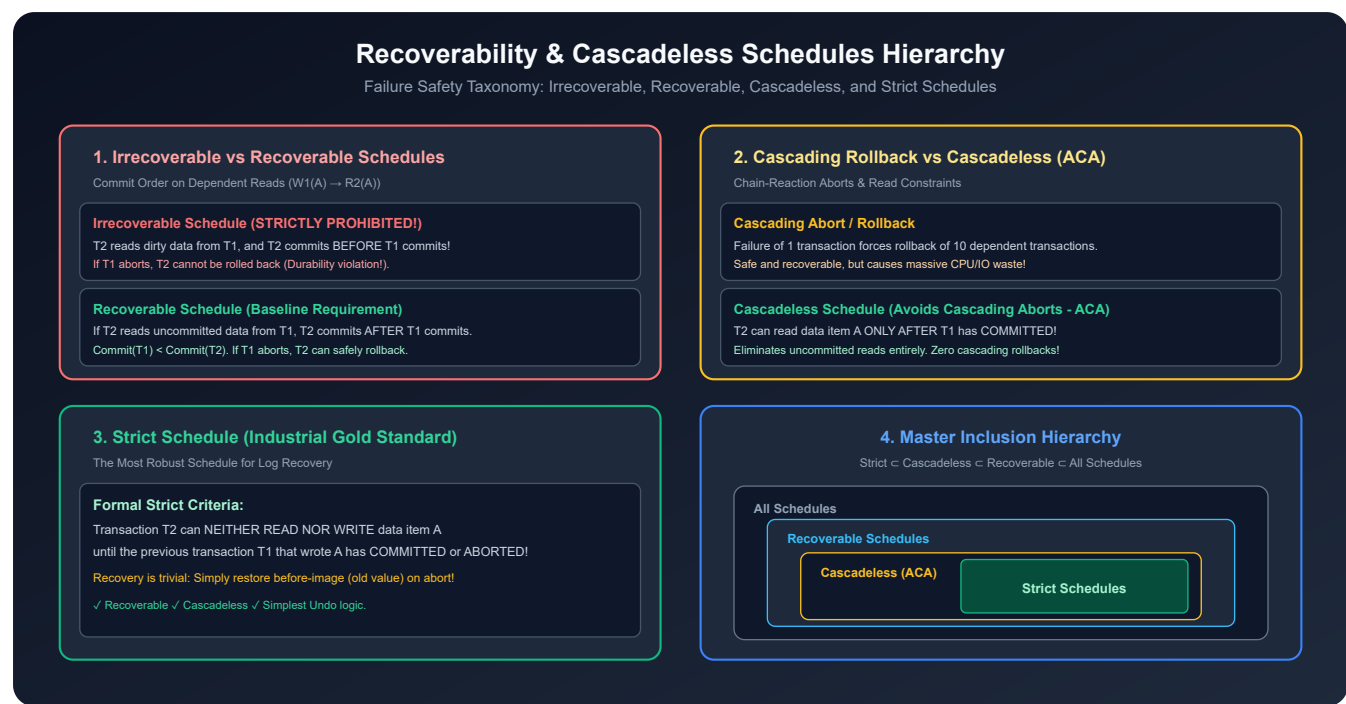
3. **Final Write:** In SS , T_3 performs final write. In Candidate 2, T_3 performs final write. **(Satisfied!)**

Final Verdict:

Schedule S is view equivalent to the serial schedule $\langle T_1, T_2, T_3 \rangle$.

Therefore, schedule S is **VIEW SERIALIZABLE**!

Module 06: Recoverability & Cascadeless Schedules



> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Irrecoverable Schedules (Hazard), Recoverable Schedules, Cascading Rollback vs Cascadeless (ACA), Strict Schedules, Master Inclusion Hierarchy, Solved Schedule Testing.

1. Why Do We Need Recoverable Schedules?

Serializability yeh ensure karti hai ki concurrent schedule ka outcome kisi serial execution ke barabar ho. Lekin Serializability **Failures (Aborts)** ko consider nahi karti!

Real-world database me transactions network drop, disk error, ya divide-by-zero ki wajah se **Abort** hoti hain. Agar ek abort hui transaction ka data kisi doosri transaction ne read kar liya ho aur wo commit ho chuki ho, toh system aisi state me phans jaata hai jahan se **database ko recover karna mathematically impossible ho jaata hai!**

Isliye relational DBMS me **Recoverability** serializability se bhi zyada critical property hoti hai!

2. Classification of Schedules Based on Recovery

2.1 Irrecoverable Schedule (Strictly Prohibited!)

Definition: Jab transaction T_2 data item A ko read karta hai jise T_1 ne update kiya tha ($W_1(A)$

$\rightarrow R_2(A)$), aur **T_2 pehle COMMIT ho jaata hai jabki T_1 abhi active tha.**

Uske baad T_1 fail hokar **Abort** ho jaata hai!

Hazard: ACID properties ke Durability rule ke according hum committed transaction T_2 ko rollback nahi kar sakte, aur Atomicity ke according hum T_1 ke uncommitted data ko database me chhod nahi sakte!

Verdict: Irrecoverable schedules database corrupt karte hain aur kisi bhi RDBMS me allow nahi hote.

T1 T2

| |

Write(A) |

| Read(A) <-- Reads uncommitted A

| Commit <-- T2 COMMITS!

[ABORT / CRASH] | <-- T1 aborts! T2 cannot rollback!

,

2.2 Recoverable Schedule (Baseline Requirement)

Definition: Agar transaction T_2 uncommitted data item A read karta hai jo T_1 ne likha tha, toh T_2 ka commit operation compulsory roop se **T_1 ke commit hone ke BAAD hona chahiye:**

$$\mathbf{ext\{Commit\}(T_1) < ext\{Commit\}(T_2)}$$

Agar T_1 abort hota hai, toh T_2 abhi commit nahi hua hai, isliye T_2 ko bhi safe tarike se rollback kiya ja sakta hai!

2.3 Cascading Rollback / Cascading Abort

Ek recoverable schedule me agar transaction T_1 fail ho jaye, toh un sabhi transactions ($T_2, T_3, \dots, T_k$) ko bhi abort aur rollback karna padta hai jinhone T_1 ka uncommitted data padha tha!

Yeh chain reaction bohot saara CPU aur I/O time waste karti hai.

2.4 Cascadeless Schedule (Avoids Cascading Aborts - ACA)

Rule: Transaction T_2 kisi data item A ko **tabhi read kar sakta hai jab use likhne wala transaction T_1 already COMMIT ho chuka ho!**

Dirty reads physically allow hi nahi hote!

Agar T_1 abort hota hai, toh kisi aur transaction ko rollback karne ki zaroorat nahi padti (Zero cascading rollbacks).

2.5 Strict Schedule (Industrial Standard)

Rule: Transaction T_2 kisi data item A ko **na toh READ kar sakta hai aur na hi WRITE kar sakta hai** jab tak use pehle likhne wala transaction T_1 commit ya abort na ho jaye!

Huge Advantage in Crash Recovery: Agar koi transaction abort hoti hai, toh database recovery manager ko koi complex calculation nahi karni padti; wo seedha log file se **Before-Image (Purani value)** uthakar cell me paste kar deta hai!

3. Master Inclusion Hierarchy

$\mathbf{\text{Strict}} \subseteq \text{Cascadeless (ACA)} \subseteq \text{Recoverable} \subseteq \text{All Schedules}$

Har Strict schedule by default Cascadeless hota hai.

Har Cascadeless schedule by default Recoverable hota hai.

Lekin har Recoverable schedule Cascadeless nahi hota!

4. Architectural Diagram

5. Gateway Classes Solved AKTU Examination Problem

Question: Test whether the following schedule SS is Recoverable, Cascadeless, or Strict:

$SS: R_1(X); \quad R_2(Z); \quad W_1(X); \quad R_2(Y); \quad W_3(Y); \quad R_2(X); \quad W_2(Z); \quad W_2(Y);$
 $\quad W_1(Z); \quad C_1; \quad C_2; \quad C_3$

Step-by-Step Analysis:

- Find all Dirty Reads ($W_i \rightarrow R_j$):**
 - $W_1(X)$ happens, followed by $R_2(X)$ before T_1 commits!
 - This is a Dirty Read from T_1 to T_2 .

2. Check Commit Order:

- T_1 commits at step C_1 .
- T_2 commits at step C_2 .
- Here $C_1 < C_2$ (T_1 commits before T_2).
- Therefore, the schedule is **RECOVERABLE**!

3. Check Cascadeless:

- In Cascadeless, a transaction can only read AFTER commit.
- But T_2 read X at $R_2(X)$ while T_1 was uncommitted!
- Therefore, schedule SS is **NOT Cascadeless**!

4. Check Strict:

- Since it is not cascadeless, it can **NEVER be Strict**!

Conclusion: Schedule SS is **Recoverable with Cascading Rollback**, but **NOT Cascadeless and NOT Strict**!

Module 07: Concurrency Control & 2PL Protocols

Concurrency Control Protocols & Two-Phase Locking (2PL)

Shared/Exclusive Locks, Growing & Shrinking Phases, Strict 2PL & Rigorous 2PL

1. Two-Phase Locking (2PL) Mechanics

Mathematical Guarantee: 2PL Ensures Conflict Serializability!

Phase 1: Growing Phase (Expanding)

- Transaction can **ACQUIRE** locks (Shared or Exclusive).
- Transaction **CANNOT** release any lock during this phase!

Lock conversion: Upgrades (S → X) allowed.

★ LOCK POINT: Instant when final lock is acquired

Phase 2: Shrinking Phase (Contracting)

- Transaction can **RELEASE** locks.
- Transaction **CANNOT** acquire ANY new lock once shrinking starts!

Lock conversion: Downgrades (X → S) allowed.

Critical 2PL Limitations:

- May suffer from **DEADLOCKS** (circular lock wait).
- Basic 2PL may cause **CASCADING ROLLBACKS** (requires Strict 2PL).

2. Lock Compatibility & 2PL Variants

Shared vs Exclusive Locks & Industrial Enhancements

Lock Compatibility Matrix:

- Shared Lock S(A): Compatible with another S(A). [Read-Read OK]
- Exclusive Lock X(A): Incompatible with BOTH S(A) and X(A)!

If T1 holds X(A), any request for S(A) or X(A) by T2 **MUST WAIT**.

Strict 2PL (Prevents Cascading Aborts)

All Exclusive Locks X(A) are held until COMMIT / ABORT!

Guarantees Strict Recoverability. Eliminates dirty reads!

Rigorous 2PL (Serial Order = Commit Order)

ALL locks (both Shared and Exclusive) are held until COMMIT!

Transactions serialize in exact commit order. Zero dirty reads.

Conservative 2PL (Static 2PL)

Transaction locks ALL required items before starting execution.

✓ 100% Deadlock-Free! ✗ Hard to predict needed items in advance.

> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Concurrency Control Need, Shared & Exclusive Locks, Basic Two-Phase Locking (2PL), Growing vs Shrinking Phases, Lock Point, Strict 2PL, Rigorous 2PL, Conservative 2PL, Thomas' Write Rule.

1. Why Do We Need Locking Protocols?

Precedence Graph runtime par dynamic concurrent systems ke andar har query ke liye graph draw karke cycles check nahi kar sakta (too high computational overhead).

Isliye DBMS **Locking Protocols** use karta hai jisme data items access karne se pehle transactions ko **Lock** lena padta hai aur kaam khatam hone par **Unlock** karna padta hai.

2. Lock Modes & Compatibility

Mode	Symbol	Meaning	Compatibility
Shared Lock	\$\$\$	**Read Lock:** Sirf data ko read karne ke liye liya jaata hai.	Multiple transactions simultaneously \$\$\$ lock le sakti hain (\$S-\$S compatible).
Exclusive Lock	\$X\$	**Write Lock:** Data ko read aur update dono karne ke liye liya jaata hai.	Koi doosri transaction na \$\$\$ le sakti hai na \$X\$ (\$X-\$S\$ and \$X-X\$ incompatible).

3. Two-Phase Locking (2PL) Protocol

2PL protocol ensure karta hai ki schedule **Conflict Serializable** ho. Isme transaction do strict phases me divide hoti hai:

Number of

Locks Held

▲ Lock Point

| ▲

| / — \

| / \

| / \

| / \

| Growing Phase Shrinking Phase

| (Only Locks Acquired) (Only Locks Released)

_____→ Time

1. Growing Phase (Expanding Phase):

- Transaction naye locks acquire kar sakti hai.
- Transaction koi bhi lock release **nahi** kar sakti!

2. Lock Point:

- Wo specific instant of time jab transaction ne apna aakhiri lock acquire kiya.

3. Shrinking Phase (Contracting Phase):

- Transaction purane locks release kar sakti hai.
- Transaction koi bhi naya lock acquire **nahi** kar sakti!

Notice:
Fundamental Theorem of 2PL:
Agar koi schedule 2PL protocol follow karta hai, toh wo ****guaranteed Conflict Serializable hoga****!

4. Variations of 2PL (AKTU Exam Comparison)

2PL Variant	Lock Release Rule	Guarantees Serializability?	Prevents Cascading Aborts?	Prevents Deadlocks?
Basic 2PL	Locks can be released in shrinking phase anytime	\$\checkmark\$	\$imes\$	\$imes\$

2PL Variant	Lock Release Rule	Guarantees Serializability?	Prevents Cascading Aborts?	Prevents Deadlocks?
Strict 2PL	Exclusive locks (\$X\$) held until Commit/Abort	\$\checkmark\$	**\$\checkmark\$ (Strict)**	\$\times\$
Rigorous 2PL	**ALL** locks (\$S\$ and \$X\$) held until Commit/Abort	\$\checkmark\$	**\$\checkmark\$ (Rigorous)**	\$\times\$
Conservative 2PL	Acquires all locks upfront before start	\$\checkmark\$	\$\times\$	**\$\checkmark\$ (Deadlock Free!)**

5. Architectural Diagram

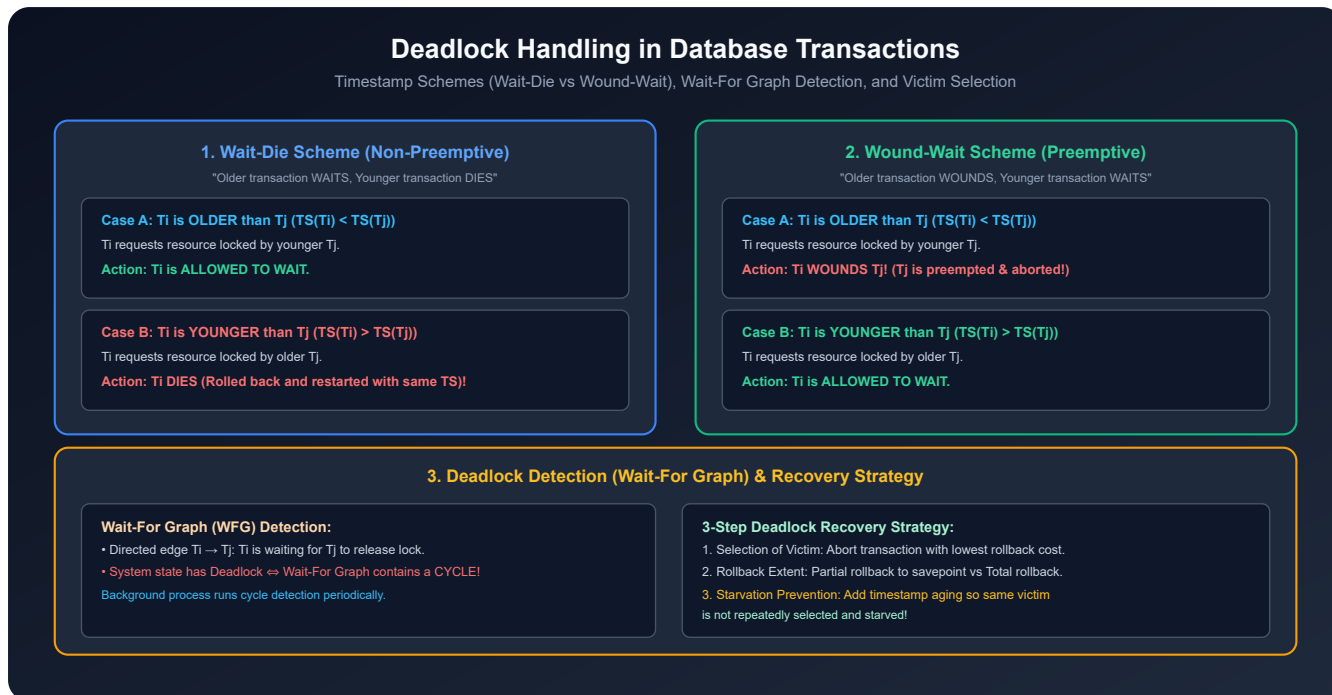
6. Solved AKTU Examination Question (10 Marks)

Question: What is Two-Phase Locking (2PL)? Differentiate between Strict 2PL and Rigorous 2PL. Does basic 2PL ensure freedom from deadlocks? Justify. *(AKTU 2019-20, 2022-23)*

Solution Summary:

1. Definition of Growing and Shrinking phases with the Lock Point graph.
2. Proof that 2PL guarantees conflict serializability (by contradiction: cycles in serialization graph require a transaction to acquire a lock after releasing one, violating 2PL).
3. Contrast Strict 2PL vs Rigorous 2PL (Strict releases Shared locks early; Rigorous holds ALL locks until commit).
4. Freedom from deadlocks: **NO**, Basic 2PL does NOT prevent deadlocks! Provide counter-example (\$T_1\$ holds lock on \$A\$ and requests \$B\$; \$T_2\$ holds lock on \$B\$ and requests \$A\$, creating a cyclic wait).

Module 08: Deadlock Handling in Transactions



> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Deadlock Definition, Timestamp-Based Prevention (Wait-Die vs Wound-Wait), Wait-For Graph (WFG) Detection, Recovery from Deadlocks (Victim Selection, Rollback, Starvation Prevention).

1. What is a Deadlock in Database Systems?

Jab do ya do se zyada transactions ek doosre ke dwara hold kiye gaye locks ke release hone ka circular wait karti hain, toh system ek frozen state me chala jaata hai jahan koi bhi transaction aage proceed nahi kar sakti. Is condition ko **Deadlock** kehte hain.

Example:

- T_1 holds exclusive lock on A , requests lock on B .
- T_2 holds exclusive lock on B , requests lock on A .
- Neither can proceed $\Rightarrow$ Deadlock!

2. Deadlock Prevention: Timestamp Schemes

Deadlock hone se pehle hi prevent karne ke liye DBMS transactions ko monotonic timestamps assign karta hai ($TS(T_i)$). Chhota timestamp matlab **Older (Senior)** transaction!

Suppose transaction T_i requests a lock held by transaction T_j :

2.1 Wait-Die Scheme (Non-Preemptive)

Rule:

- Agar T_i **Older** hai ($TS(T_i) < TS(T_j)$) $\implies T_i$ is allowed to **WAIT**.
- Agar T_i **Younger** hai ($TS(T_i) > TS(T_j)$) $\implies T_i$ **DIES** (rolled back and restarted with same timestamp).

Mnemonic: "Older waits, younger dies".

2.2 Wound-Wait Scheme (Preemptive)

Rule:

- Agar T_i **Older** hai ($TS(T_i) < TS(T_j)$) $\implies T_i$ **WOUNDS** T_j (T_j is aborted and forced to release lock).
- Agar T_i **Younger** hai ($TS(T_i) > TS(T_j)$) $\implies T_i$ is allowed to **WAIT**.

Mnemonic: "Older wounds, younger waits".

2.3 Wait-Die vs Wound-Wait Comparison (AKTU Exam Favorite)

Parameter	Wait-Die Scheme	Wound-Wait Scheme
Preemption	Non-preemptive (Locks never taken away)	Preemptive (Older transaction preempts younger)
Number of Rollbacks	High (Younger transaction repeatedly aborts)	Much Lower (Younger waits quietly)
Lock Holding	Resources remain held longer	Quick release upon wound
Restart Policy	Restarts with original timestamp to prevent starvation	Restarts with original timestamp

3. Deadlock Detection: Wait-For Graph (WFG)

Agar prevention use nahi kiya jaata, toh DBMS transactions ko freely lock lene deta hai aur periodic background thread chala kar deadlock detect karta hai:

Wait-For Graph $G = (V, E)$:

- Vertices V : All active transactions.
- Directed edge T_i

$\rightarrow T_j$: T_i is waiting for a lock held by T_j .

Theorem: Deadlock exists in the system **if and only if the Wait-For Graph contains a Directed Cycle!**

4. Deadlock Recovery Strategy

Jab Wait-For Graph me cycle detect hoti hai, toh system ko deadlock break karne ke liye 3 steps lene padte hain:

1. Selection of Victim:

- Kisi ek transaction ko abort karke locks release karwana.
- Victim chunne ke criteria: Jisme sabse kam operations execute hue hon, jiska rollback cost minimum ho, jo kam resources hold kar rahi ho.

2. Rollback Extent:

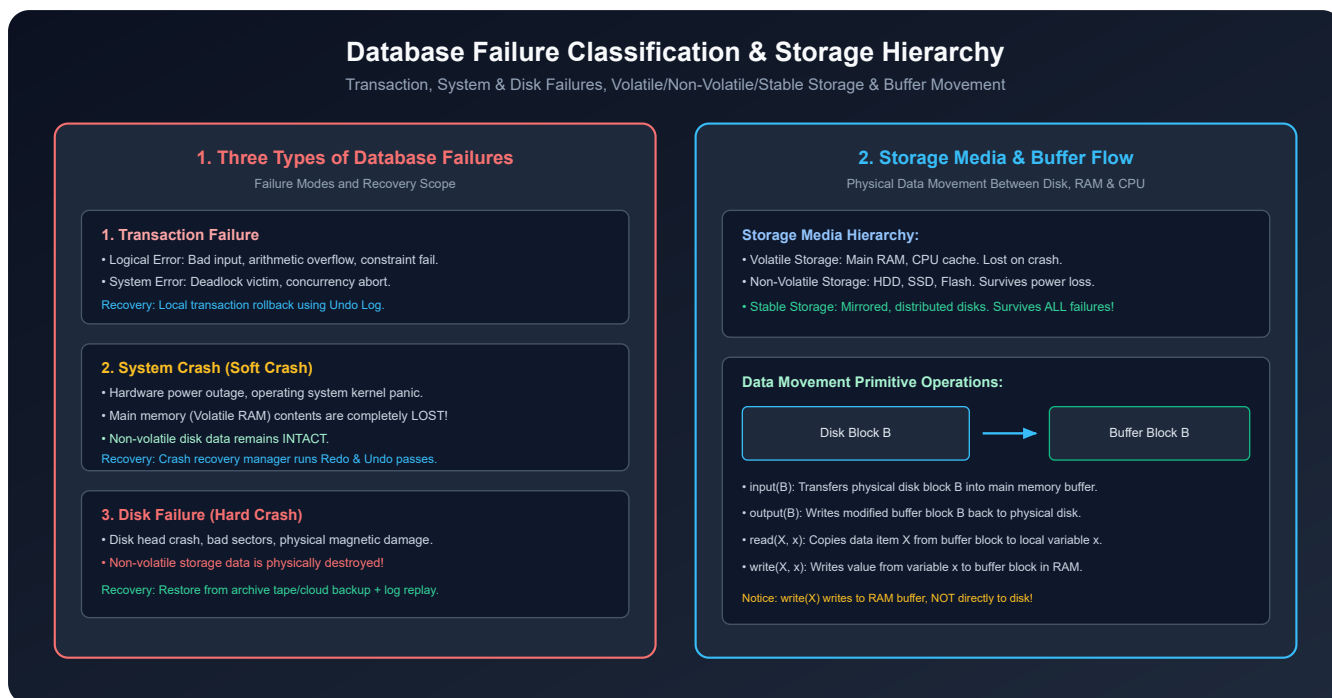
- **Total Rollback:** Transaction ko start se abort karna.
- **Partial Rollback (Savepoints):** Transaction ko sirf us point tak undo karna jahan tak deadlock break ho jaye.

3. Starvation Prevention:

- Agar ek hi transaction baar-baar victim ban kar abort hoti rahegi, toh wo kabhi finish nahi ho payegi (**Starvation**).
- **Solution:** Har baar abort hone par transaction ka priority weight badha diya jaata hai (Aging) taaki wo dobara victim na chuni jaye.

5. Architectural Diagram

Module 09: Failure Types & Storage Structures



> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Failure Classification (Transaction, System Crash, Disk Failure), Storage Media Hierarchy (Volatile, Non-Volatile, Stable Storage), Buffer Management and Primitive Data Movement Operations (`input`, `output`, `read`, `write`).

1. Classification of Database Failures

Database system me failures ko unke scope aur loss ke according 3 categories me divide kiya jaata hai:

1.1 Transaction Failure

Logical Error: Transaction business logic ya application error ki wajah se aage nahi badh sakti (e.g., negative balance check, divide-by-zero, foreign key violation).

System Error: Concurrency control manager deadlock break karne ke liye transaction ko victim select karke abort kar deta hai.

Recovery: Local transaction rollback through undo log.

1.2 System Crash (Soft Failure)

Cause: Power failure, OS kernel panic, hardware motherboard crash.

Consequence: Main memory (RAM) ka poora volatile content destroy ho jaata hai. Disk par store data safe rehta hai.

Assumption (Fail-Stop Model): Hardware crash hone par database hardware turant halt ho jaata hai bina disk data ko corrupt kiye.

1.3 Disk Failure (Hard Failure)

Cause: Disk read/write head crash, magnetic degradation, bad disk blocks.

Consequence: Non-volatile storage permanently damage ho jaata hai.

Recovery: Periodic archival database backups (Dump) aur stable log copies se full restore.

2. Storage Media Hierarchy

1. Volatile Storage:

- RAM, cache memory. Extremely fast ($O(1)$ ns access). Lost during power cuts.

2. Non-Volatile Storage:

- Magnetic hard disks, solid-state drives (SSD), flash memory. Retains data across crashes.

3. Stable Storage:

- Theoretical ideal storage jo **kisi bhi failure me kabhi corrupt nahi hota**.
- Physically implemented using **RAID 1 / Mirroring** across independent disk controllers aur geographically remote backup sites.

3. Data Movement and Buffer Management Architecture

Database disk par block size me store hota hai (e.g., 4KB blocks). Transaction CPU registers me execute hoti hai. Inke beech data ka flow 4 fundamental primitive operations se hota hai:

,

[Physical Disk]

▲ |

output | | input

| ▼

[Main Memory Buffer]

▲ |

write | | read

| ▼

[Transaction Local Registers]

,

input(B): Disk block B ko read karke memory buffer block me laata hai.

read(X, x): Buffer block se item X ko transaction ke local variable x me copy karta hai.

write(X, x): Local variable x ki value ko RAM ke buffer block me likhta hai. *(Caution: Yeh abhi disk par nahi gaya hai!)*

output(B): RAM ke dirty buffer block B ko disk par permanently flush karta hai.

4. Architectural Diagram

Module 10: Log-Based Recovery & WAL Protocol

Log-Based Recovery & Write-Ahead Logging (WAL)

Log Record Anatomy, WAL Invariant, Deferred vs Immediate Modification & Undo/Redo Logic

The Write-Ahead Logging (WAL) Sacred Protocol:

- Before database page on disk is updated with new value V_{new} , the log record MUST be written to stable storage!
- Before $\langle Ti \text{ commit} \rangle$ log record is confirmed to user, ALL preceding log records for Ti MUST be flushed to disk.

Guarantee: If crash occurs mid-flight, recovery manager always has log history to Undo or Redo!

1. Deferred Database Modification

"No-Undo / Redo-Only" Recovery Policy

Mechanism:

- Physical disk updates are POSTPONED until AFTER commit!
- While active, writes are recorded ONLY in log file and memory.

Log records only need NEW value: $\langle Ti, X, V_{new} \rangle$

Crash Recovery Action:

- If log has $\langle Ti \text{ start} \rangle$ and $\langle Ti \text{ commit} \rangle \Rightarrow \text{REDO}(Ti)$.
- If log has $\langle Ti \text{ start} \rangle$ but NO commit $\Rightarrow$ NO ACTION NEEDED!
(Since uncommitted data was never written to disk!).

✓ Zero Undo overhead! ✗ Requires large memory buffers.

2. Immediate Database Modification

"Undo / Redo" Standard Enterprise Policy

Mechanism:

- Database disk can be updated while transaction is ACTIVE.
- Disk may contain uncommitted "dirty" updates at any time.

Log records MUST have BOTH values: $\langle Ti, X, V_{old}, V_{new} \rangle$

Crash Recovery Action:

- If log has $\langle Ti \text{ start} \rangle$ and $\langle Ti \text{ commit} \rangle \Rightarrow \text{REDO}(Ti)$.
- If log has $\langle Ti \text{ start} \rangle$ but NO commit $\Rightarrow \text{UNDO}(Ti)$!
(Restores $X = V_{old}$ to eliminate dirty uncommitted changes).

Idempotency Guarantee: $\text{Redo}(\text{Redo}) = \text{Redo}$, $\text{Undo}(\text{Undo}) = \text{Undo}$.

> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Log Record Architecture, Write-Ahead Logging (WAL) Protocol, Deferred Database Modification vs Immediate Database Modification, Undo and Redo Algorithms, Idempotency.

1. What is a Database Log?

Database **Log** stable storage par likha gaya ek append-only audit trail hota hai jisme transaction execution ke har operation ka chronological record rehta hai.

Har log record ka structure:

- $\langle T_i \text{ ext} \{ \text{start} \} \rangle$

angle\$: Transaction T_i shuru hua.

- $\langle T_i, X, V_{old}, V_{new} \rangle$

angle\$: T_i ne data item X ko purani value V_{old} (before-image) se badal kar nayi value V_{new} (after-image) kar diya.

- $\langle T_i \text{ ext} \{ \text{commit} \} \rangle$

angle\$: T_i successfully commit hua.

- $\langle T_i \text{ ext} \{ \text{abort} \} \rangle$

angle\$: T_i abort hua aur rollback complete hua.

2. Write-Ahead Logging (WAL) Protocol

Crash recovery ki durability aur atomicity ensure karne ke liye WAL protocol follow karna compulsory hai:

WAL Rule 1: Disk par data page ko new value se overwrite karne se pehle, us update ka log record non-volatile stable storage par flush ho chuka hona chahiye!

WAL Rule 2: User ko commit acknowledgment bhejne se pehle, us transaction ke saare log records stable storage par physically written hone chahiye!

3. Deferred vs Immediate Database Modification

Parameter	Deferred Modification (NO UNDO)	Immediate Modification (UNDO & REDO)
When are Disk Pages Updated?	Only AFTER commit	Immediately while active
Log Record Contents		

Both old & new: $\langle T_i, X, V_{old}, V_{new} \rangle$

Crash Action for Active Uncommitted	No action! (Disk was never touched)	UNDO (Restore $X = V_{old}$)
Crash Action for Committed	REDO ($X = V_{new}$)	REDO ($X = V_{new}$)
Memory Buffer Pressure	High (must hold all updates in RAM)	Low (can flush dirty pages anytime)
Industry Adoption	Rarely used in high-volume systems	Standard in Oracle, MySQL, SQL Server

4. Undo and Redo Operation Primitives

extUndo(T_i): Log ko backward scan karke har update record $\langle T_i, X, V_{old}, V_{new} \rangle$

ke liye data item X ko uski purani value V_{old} set karta hai.

extRedo(T_i): Log ko forward scan karke har update record $\langle T_i, X, V_{old}, V_{new} \rangle$

ke liye data item X ko uski nayi value V_{new} set karta hai.

Idempotence Property: Agar recovery process ke dauran dubara system crash ho jaye, toh Undo aur Redo ko kitni bhi baar repeat run kiya ja sakta hai bina koi galat outcome generate kiye!

$$\text{extRedo}(\text{extRedo}(T_i)) = \text{extRedo}(T_i)$$

5. Architectural Diagram

Module 11: Checkpoints & Shadow Paging



> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Checkpoint Mechanics, Resolving Active Transactions (Undo-List & Redo-List), Shadow Paging Technique, Current vs Shadow Directory, Trade-offs.

1. Why Do We Need Checkpoints?

Normal Log-based recovery me agar system crash hota hai, toh recovery manager ko log file ke shuru se aakhiri tak poora scan karna padta hai.

Ek enterprise server jo 6 mahine se chal raha hai, usme millions of transactions honge. Un sabhi ko scan karke re-execute karna ghanto (hours) ka time lega!

Checkpoint database me ek periodic milestone hota hai jahan tak ke saare changes disk par physically sync ho chuke hote hain.

Benefit: Recovery manager ko checkpoint se pehle commit ho chuki transactions ko dubara chhoone ki koi zaroorat nahi hoti!

2. Checkpoint Creation Lifecycle

,

1. Temporary Freeze: New transaction start operations are queued.
2. Buffer Flush: RAM ke sabhi modified (dirty) pages ko disk par write kiya jaata hai.
3. Checkpoint Record: Log me record likha jaata hai:
4. Log Flush: Log file ko stable storage par flush kiya jaata hai.

5. Resume: Normal execution continues.

,

3. Recovery Algorithm with Checkpoints (AKTU Solved Pattern)

Jab crash hota hai, recovery manager log ko end se scan karte hue peeche aata hai:

1. Sabse recent checkpoint record dhundo: $\langle \text{checkpoint} \rangle L$

angle\$.

2. Do empty sets banao:

- **Undo-List** $= L$ (Sabhi transactions jo checkpoint ke waqt chal rahi thi).

- **Redo-List** $= \emptyset$.

3. Checkpoint se aage (forward) scan karo:

- Agar kisi transaction ka $\langle T_i \rangle \text{ commit}$

angle\$ record milta hai $\implies T_i$ ko Undo-List se nikaal kar **Redo-List** me daal do!

- Agar checkpoint ke baad kisi transaction ka $\langle T_j \rangle \text{ start}$

angle\$ milta hai $\implies T_j$ ko **Undo-List** me add kar do!

4. **Action:**

- Redo-List ki sabhi transactions ko **REDO** karo (forward replay).

- Undo-List ki sabhi transactions ko **UNDO** karo (backward rollback).

4. Shadow Paging Technique

Shadow paging ek **Log-Free Recovery** architecture hai jo RDBMS me transaction recovery simplify karta hai:

,

[In-Memory Current Page Table] $\longrightarrow$ Points to Newly Written Disk Blocks

[On-Disk Shadow Page Table] $\longrightarrow$ Points to Original Clean Disk Blocks

,

1. **Working:**

- Transaction start hote waqt Shadow Page Table ko copy karke Current Page Table banaya jaata hai.

- Har write operation ek naye disk block par hota hai, aur Current Page Table us naye block ko point karta hai.

- Shadow Page Table disk par unmodified rehti hai.

2. Commit:

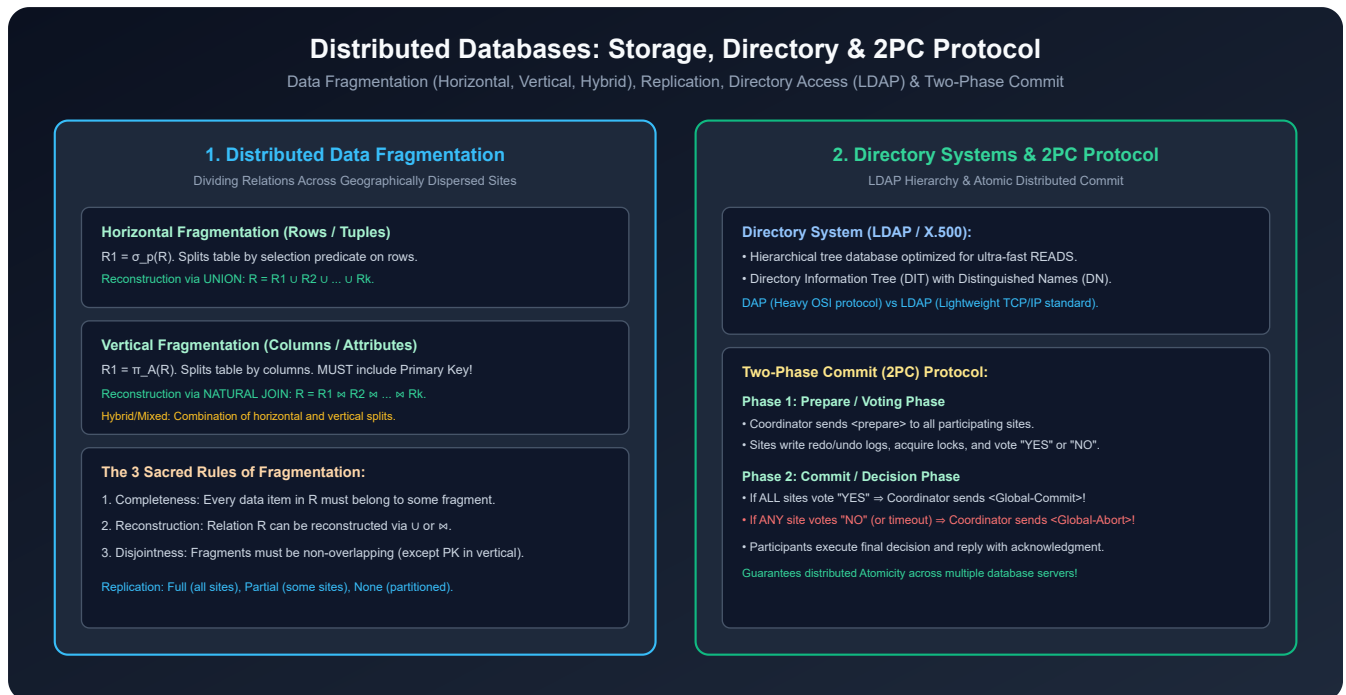
- Current Page Table ka pointer disk ke root pointer par atomically swap kar diya jaata hai.
- Current table ab nayi Shadow Table ban jaati hai!

3. Crash / Abort:

- RAM ki Current Page Table ko discard kar do!
- Disk par Shadow Page Table pehle se intact hai, isliye zero undo operations me system recover ho jaata hai!

5. Architectural Diagram

Module 12: Distributed Databases & Directory Systems



> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Distributed DBMS Architecture, Data Fragmentation (Horizontal, Vertical, Hybrid), Rules of Fragmentation, Data Replication, Directory Systems (LDAP, X.500, DIT), Two-Phase Commit (2PC) Protocol.

1. What is a Distributed Database (DDBMS)?

Distributed Database multiple interconnected physical sites (nodes) ka collection hota hai jo network ke through communicate karte hain aur user ko single coherent database ki tarah appear hote hain (**Distribution Transparency**).

Homogeneous DDBMS: Sabhi sites par same database software (e.g., all Oracle) aur same data models hote hain.

Heterogeneous DDBMS: Alag-alag sites par alag DBMS hote hain (e.g., Site 1 on PostgreSQL, Site 2 on MySQL).

2. Distributed Data Storage Techniques

2.1 Data Fragmentation

Ek relation R ko multiple logical sub-parts me divide karna:

1. Horizontal Fragmentation:

- Relation ko rows (tuples) me divide karta hai using Selection operator $\sigma_p(R)$.

- Reconstruction: **Union** ($\cup$) operation.

- Example: $R_{\{ \text{Delhi} \}} = \sigma_{\{ \text{City} = 'Delhi' \}}(R)$ and $R_{\{ \text{Mumbai} \}} = \sigma_{\{ \text{City} = 'Mumbai' \}}(R)$.

2. Vertical Fragmentation:

- Relation ko columns (attributes) me divide karta hai using Projection operator $\pi_A(R)$.
- **Crucial Rule:** Primary Key attribute har fragment me present hona **mandatory** hai!
- Reconstruction: **Natural Join** ($\bowtie$) operation.

3. Hybrid / Mixed Fragmentation:

- Horizontal aur Vertical fragmentation ka combination.

2.2 The Three Sacred Rules of Fragmentation

1. **Completeness:** Original table ka har single data item kisi na kisi fragment me zaroor appear hona chahiye.
2. **Reconstruction:** Fragments ko combine karke original relation bina data loss ke wapas banayi ja sake ($\cup$ for horizontal, $\bowtie$ for vertical).
3. **Disjointness:** Fragments non-overlapping hone chahiye (vertical me sirf Primary Key duplicate hoti hai).

2.3 Data Replication

Full Replication: Relation ki identical copy har single network site par store ki jaati hai (Maximum read availability, expensive updates).

Partial Replication: Copies sirf un sites par rakhi jaati hain jahan query frequency high ho.

No Replication: Har fragment sirf ek single site par rehta hai.

3. Directory Systems (LDAP & X.500)

Directory Service: Ek specialized hierarchical database jo **high-frequency READS** aur low-frequency writes ke liye optimize hota hai (e.g., user authentication, phone books, DNS).

Directory Information Tree (DIT): Tree structure jisme nodes **Distinguished Names (DN)** se uniquely identify hoti hain (e.g., cn=Shivam,ou=Engineering,dc=example,dc=com).

DAP vs LDAP:

- **DAP (Directory Access Protocol):** Heavyweight OSI protocol stack.
- **LDAP (Lightweight DAP):** Modern lightweight standard jo directly TCP/IP par run hota hai.

4. Two-Phase Commit (2PC) Protocol

Distributed system me sabhi sites par atomicity maintain karne ke liye **Two-Phase Commit Protocol** use hota hai:

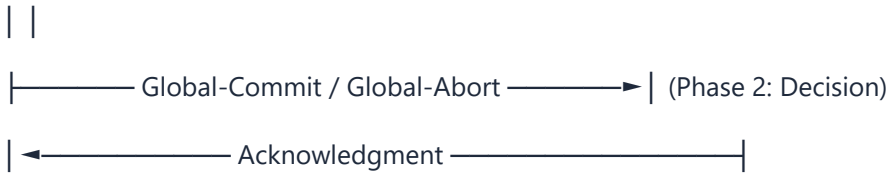
,

Coordinator Participants (Sites 1..N)

| |

|————— Prepare? —————> | (Phase 1: Voting)

| <————— Vote YES / NO ————— | (Locks held & Logged)



1. Phase 1: Prepare / Voting Phase:

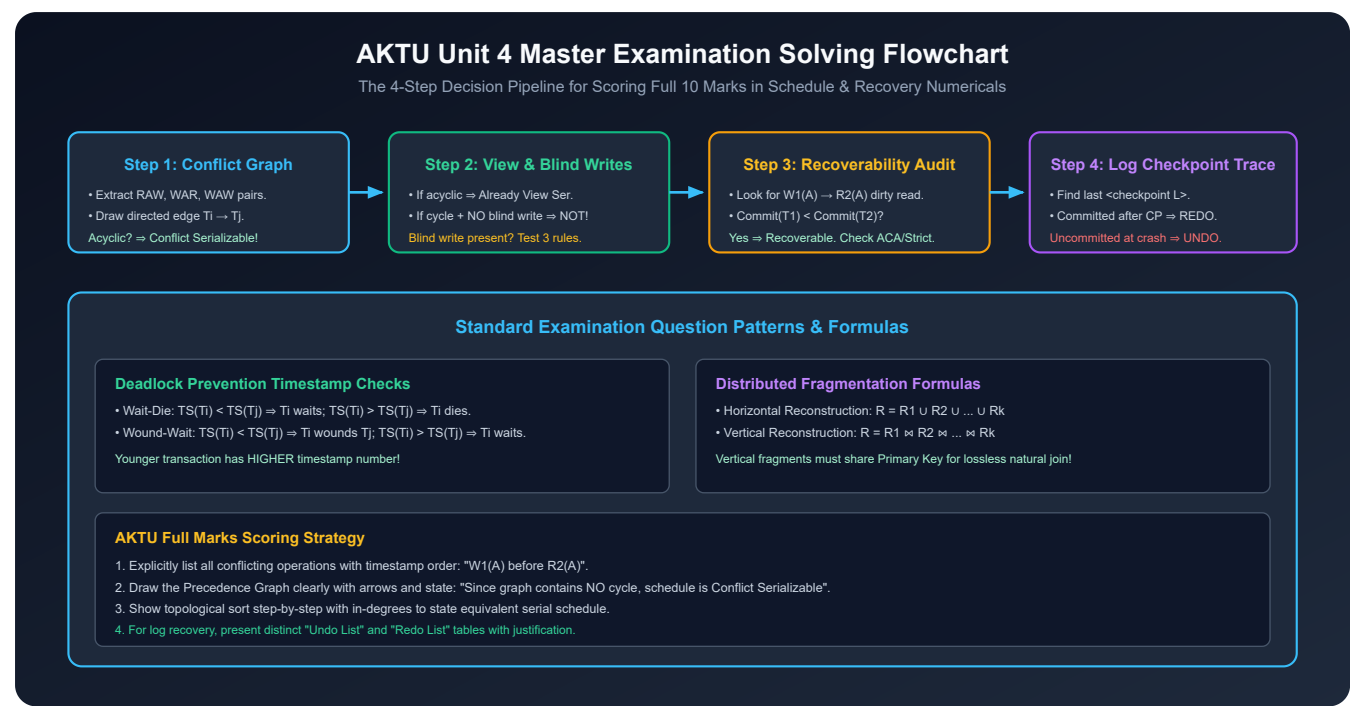
- Coordinator sabhi participants ko Prepare message bhejta hai.
- Participants apne local changes log me write karte hain aur VOTE_COMMIT ya VOTE_ABORT reply karte hain.

2. Phase 2: Commit / Decision Phase:

- Agar **SABHI** participants ne YES vote kiya $\implies$ Coordinator GLOBAL_COMMIT bhejta hai.
- Agar kisi **EK** participant ne bhi NO vote kiya ya timeout hua $\implies$ Coordinator GLOBAL_ABORT bhejta hai!
- Sabhi sites decision apply karke acknowledgment return karti hain.

5. Architectural Diagram

Module 13: AKTU PYQs & Solved Schedules



- On $W_3(Y)$ is followed by $W_2(Y) \implies \mathbf{T_3} \rightarrow \mathbf{T_2}$ (Already present).
- On $R_1(Z)$ is followed by $W_2(Z) \implies \mathbf{T_1} \rightarrow \mathbf{T_2}$.

Precedence Graph Edges:

$\mathbf{T_3}$

$\rightarrow \mathbf{T_1}, \quad \mathbf{T_3}$

$\rightarrow \mathbf{T_2}, \quad \mathbf{T_1}$

$\rightarrow \mathbf{T_2}$

Cycle Check: No cycles exist in the graph (strictly Acyclic Directed Graph).

Topological Sorting:

- In-degree of $\mathbf{T_3} = 0 \implies \mathbf{T_3}$ is first.
- In-degree of $\mathbf{T_1} = 0$ (after removing $\mathbf{T_3}$) $\implies \mathbf{T_1}$ is second.
- Remaining: $\mathbf{T_2}$ is third.

Equivalent Serial Schedule: $\mathbf{T_3}$

$\rightarrow \mathbf{T_1}$

$\rightarrow \mathbf{T_2}$.

Verdict 1: Schedule SS is **Conflict Serializable!**

2. Recoverability Test

Look for Dirty Reads (W_i

$\rightarrow R_j$):

- $W_3(Y)$ is followed by $R_2(Y)$ while $\mathbf{T_3}$ is uncommitted.
- So $\mathbf{T_2}$ reads uncommitted data from $\mathbf{T_3}$.

Check commit orders:

- $\mathbf{T_2}$ commits at step C_2 .
- $\mathbf{T_3}$ commits at step C_3 .
- Notice: $C_2 < C_3$! That means **$\mathbf{T_2}$ COMMITS BEFORE $\mathbf{T_3}$ COMMITS!**

If $\mathbf{T_3}$ were to abort, $\mathbf{T_2}$ cannot be rolled back!

Verdict 2: Schedule SS is **IRRECOVERABLE!**

3. Cascadeless Test

In a cascadeless schedule, no transaction can read uncommitted data.

Since T_2 reads Y from T_3 before T_3 commits, dirty read exists.

Verdict 3: Schedule SS is **NOT Cascadeless!**

2. AKTU PYQ 2 (10 Marks — 2021-22 End-Sem)

Question:

Explain Log-Based Recovery with Immediate Database Modification. Suppose a system crashes with the following log contents:

,
,

What actions will the recovery manager take during crash recovery? State the Undo-List and Redo-List.

Solution:

1. Locate Checkpoint:

- Checkpoint is $\langle \text{checkpoint} \rangle \{T_1, T_2\}$

$\angle$.

- Active transactions at checkpoint: $\{T_1, T_2\}$.

2. Initialize Lists:

- $\text{Undo-List} = \{T_1, T_2\}$

- $\text{Redo-List} = \emptyset$

3. Forward Scan from Checkpoint to Crash:

- $\langle T_3 \text{ start} \rangle$

$\angle \implies$ Add T_3 to Undo-List: $\text{Undo-List} = \{T_1, T_2, T_3\}$.

- $\langle T_1 \text{ commit} \rangle$

$\angle \implies$ Move T_1 to Redo-List: $\text{Redo-List} = \{T_1\}$, $\text{Undo-List} = \{T_2, T_3\}$.

- $\langle T_2 \text{ commit} \rangle$

$\angle \implies$ Move T_2 to Redo-List: $\text{Redo-List} = \{T_1, T_2\}$, $\text{Undo-List} = \{T_3\}$.

4. Final Lists at Crash Point:

- $\text{Redo-List} = \{T_1, T_2\}$ (Both started and committed).

- $\text{Undo-List} = \{T_3\}$ (Started, updated D , but never committed before crash).

5. Recovery Actions Performed:

- **Undo Phase:** Rollback T_3 : Restore data item $D = 400$ (its old value), write $\langle T_3 \text{ abort} \rangle$

angle\$.

- **Redo Phase:** Replay \$T_1\$ and \$T_2\$: Set \$A = 100\$, \$B = 250\$, \$C = 350\$.

3. Architectural Diagram
